

Skrubifier: LLM-Assisted Conversion of Tabular ML Pipelines to the Skrub DataOps API

Tiago Frade Charansurya Udaysingh Jhurree Yining Liang

fc59795@alunos.fc.ul.pt nirvanjhurree19@gmail.com liangyn0805@gmail.com

July 20, 2026

Abstract. We present *Skrubifier*, a three-stage framework that converts tabular machine learning pipelines written in *scikit-learn*’s *Pipeline/ColumnTransformer* API into *skrub*’s *DataOps* API, using a large language model as the syntax-translation step while keeping structural pipeline understanding and correctness validation outside the LLM’s control. The framework separates concerns into an analyzer (deterministic, AST- and object-based structural extraction into a typed intermediate representation), a converter (LLM-driven translation, with an automatic static-plus-dynamic repair loop), and a validator (structural and numerical correctness checking against the original pipeline). We apply the framework to ten pipelines drawn from real MLE-Bench and Kaggle competition solutions, spanning binary/multi-class classification, regression, single-table, multi-table, and ensemble (stacking) pipeline structures. Hand-verified reference conversions pass dynamic validation on 9 of 10 pipelines; the fully automated, LLM-driven conversion pipeline passes 6 of 10 after two iterative framework fixes surfaced by evaluation itself: a validation gap in which the repair loop was wired only to catch syntactic failures and never triggered on runtime semantic errors (taking the automated pass rate from 0 of 10 to 5 of 10 once fixed), and a prompt-construction gap in which the target column name, although present in the intermediate representation, was not surfaced explicitly to the LLM (taking it from 5 to 6). We report these findings, and several other concrete bugs discovered and fixed during development, as evidence toward the framework’s robustness rather than omitting them.

1 Introduction

Kaggle [5] and MLE-Bench [4] solutions to tabular machine learning problems are overwhelmingly written against *scikit-learn*’s [1] eager *Pipeline/ColumnTransformer* API. *Skrub*’s [2] *DataOps* API instead expresses a pipeline as a lazy, build-time directed acyclic graph (DAG) of operations over *DataOps* objects (`skrub.var`, `.skb.apply`, `.skb.mark_as_X/y`, `.skb.make_learner`), enabling features such as hyperparameter search over the whole pipeline (`.skb.make_grid_search`) and, via the drop-in accelerated backend *stratum* [3], a Rust-based execution engine. Manually rewriting existing solutions into this API is tedious and error-prone; this project asks whether a large language model, appropriately constrained, can perform this conversion reliably enough to be useful.

Why not simply prompt an LLM with the source code? We found, and confirm empirically in Section 5, that this naive approach fails for three reasons. First, *DataOps* is a lazy build-time API, so the LLM must convert imperative *control flow* into DAG-construction calls, not simply rename functions. Second, real solutions mix *pandas* munging, custom functions, multiple joined tables, and cross-validation loops that have no one-to-one *DataOps* primitive. Third, correctness must be checked *numerically*: a script can be syntactically perfect *DataOps* code and still produce wrong predictions.

2 System Architecture

Skrubifier separates the conversion task into three stages, connected by a typed intermediate representation (IR), so that the LLM’s task is narrowed to syntax translation rather than open-ended program understanding.

2.1 Analyzer

The analyzer (`analyzer.py`) performs deterministic, testable structural extraction with no LLM involvement. Given a live, unfitted *scikit-learn* *Pipeline/ColumnTransformer* object, it walks the object graph recursively, recording each transformation step, the column group it applies to (inferred from *ColumnTransformer* branches), and the final estimator’s class and hyperparameters, into a *PipelineIR* dataclass (`ir.py`). For pipelines built imperatively (manual *pandas* feature engineering with no clean *Pipeline* object), an AST-based fallback captures `groupby/merge/custom-function` calls with their literal source code attached, so the converter re-implements real logic rather than approximating it. A dedicated extension, `expand_ensemble_into_steps`, was added during development after we found that *StackingRegressor/VotingClassifier*’s nested `estimators/final_estimator` parameters, not JSON-primitive values, were being silently dropped by the parameter serializer, leaving the IR blind to every base model in an ensemble; the fix expands these into individual *StepIR* entries and adds an explicit note that no single *DataOps* primitive covers stacking.

2.2 Converter

The converter (`converter.py`) is LLM-agnostic: it accepts any callable `llm.call(prompt: str) -> str` and constructs its prompt from (a) the *PipelineIR* as JSON, (b) a condensed, example-grounded *DataOps* API reference (`prompts.py`), and (c) the original source snippet for any custom steps. Two backends are wired: the GWDG/AcademicCloud SAIA endpoint [6] (OpenAI-compatible, serv-

ing open-weight models at no cost) and the Anthropic API directly. The API reference embedded in the prompt was iteratively corrected against an official skrub DataOps tutorial notebook and, later, against real runtime errors surfaced during evaluation (Section 5). For example, an early, plausible-but-incorrect belief that a function could be lifted into the DAG via `skrub.deferred(func)(dataop)` was replaced with the tutorial’s actual documented pattern, `dataop.skb.apply_func(func)`.

2.3 Validator and Repair Loop

The validator (`validator.py`) performs two checks. A *static* check parses the candidate script’s AST and confirms it actually constructs a DataOps DAG (uses the `.skb` namespace and calls `var`, `apply`, and `make_learner`) rather than emitting eager pandas/scikit-learn code with an unused `skrub` import. A *dynamic* check executes the candidate script in a subprocess against real or representative data, fits both the original and converted pipeline on the same split, and compares a task-appropriate metric within a tolerance of $\max(0.02, 0.03 \times |\text{original}|)$, loosened relative to exact equality because re-expressing a pipeline as a DAG can perturb randomized-model behavior slightly even with fixed seeds.

On failure, the specific error is fed back to the LLM for up to three repair rounds. As discussed in Section 5, an important finding of this project is that this repair loop was, for a period, wired only to the static check, meaning syntactically valid but semantically wrong scripts (hallucinated method names, incorrect DataOps operator usage) passed validation and were never repaired, despite the architecture being designed for both checks to gate acceptance.

3 Pipeline Corpus

Ten pipelines were selected freely from real, published MLE-Bench and Kaggle competition solutions (one exception, described below, is sourced from an official skrub tutorial), chosen specifically to cover a spread of dataset sizes, domains, ML task types, structural complexity, and search space size, rather than ten similar easy cases. Each pipeline is self-contained: its source script, converted DataOps script, and validation harness live in an independent directory with no shared state or cross-references to any other pipeline, so each conversion can be inspected, run, or judged entirely on its own. Table 1 summarizes the corpus.

Example 08 is a deliberate boundary case for the “tabular only” scoping requirement: its source task is entirely text classification. It is retained because the pipeline is expressed and converted as a single-column tabular DataFrame processed through `TableVectorizer`/`extttfidfVectorizer` (the same DataOps conversion mechanics used throughout the corpus) rather than a purpose-built NLP pipeline with its own tokenization or embedding infrastructure, but we flag it explicitly rather than assert an unqualified fit.

4 Bugs Found and Fixed

Dynamic validation of the ten hand-verified reference conversions surfaced seven concrete, previously-undetected bugs, five in the converted pipeline scripts and two in the framework itself. We summarize the most significant here; the complete change history with reproduction steps is maintained in the project’s `CHANGELOG.md`.

Silent prediction failure from a stateful join primitive. `skrub.Aggregator` caches aggregations computed on the training auxiliary table and reuses them at prediction time; test-set entities not present in the training auxiliary table receive NaN for every aggregated feature, causing the downstream model to degenerate to a constant-class prediction (observed AUC = 0.5 with near-identical predicted probabilities across all test rows). The source pipelines both recompute aggregations fresh at prediction time, so the correct DataOps equivalent is a stateless aggregation function invoked via `.skb.apply_func()` combined with a DataOp-level `.merge()`, which re-executes on every fit/predict call.

Feature leakage via a retained identifier column. After the above fix, one pipeline still produced chance-level performance; the cause was that an ID column (`basket_ID`), sequential between the training and test splits, remained in the feature matrix after a `drop(columns=[target])`-style selection, allowing the model to memorize the training ID range rather than learn from genuine features.

Estimator node identity confusion. During script construction, chaining `.skb.apply_func(np.expml)` directly after `.skb.apply(estimator, y=y)` raised a type error, because in the skrub version used, an estimator DataOp node evaluates to the *fitted estimator object* during `fit()`, not to its predictions; scikit-learn’s `TransformedTargetRegressor` was used instead to invert a log-target transform correctly within a single estimator node.

Library version drift. A skrub `TableVectorizer` constructor parameter was renamed between the version referenced during framework construction and the version installed at evaluation time (`high_cardinality_transformer` → `high_cardinality`), and XGBoost 3.x was found to reject the original dataset’s string-typed multi-class labels, requiring explicit integer label encoding inside the DAG. Both are disclosed as version-sensitivity risks rather than treated as one-off fixes only.

Framework-level repair-loop gap. The most significant finding, discussed further in Section 5, is architectural rather than pipeline-specific: the automated converter’s repair loop was wired only to the static (syntax/structure) check, so scripts with real runtime semantic errors were never fed back to the LLM for correction.

#	Name	Domain	Task	Complexity	Data
01	Titanic	Survival	Binary classif.	Single-table	Synthetic
02	House Prices (XGBoost)	Real estate	Regression (log target)	Single-table	Synthetic
03	Credit Fraud	Fraud detection	Binary classif.	Multi-table (2 tables)	Synthetic
04	NYC Taxi Fare	Geo/transport	Regression	Single-table, train-only filter	Real
05	Otto Group	Multi-class retail	9-class classif.	Single-table, large search space	Synthetic
06	Allstate Claims Severity	Insurance	Regression	Ensemble (stacking)	Synthetic
07	Random Acts of Pizza	Text + tabular	Binary classif.	Mixed-type single-table	Synthetic
08	Spooky Author Identification	NLP (boundary case)	3-class classif.	Single-table, text-dominant	Synthetic
09	Home Credit Default Risk	Credit scoring	Binary classif.	Multi-table (2 tables)	Synthetic
10	Santander Customer Transaction	Banking	Binary classif.	Single-table, pure numeric	Synthetic

Table 1: The ten converted pipelines. Search-space diversity is demonstrated by example 05, whose converted form uses `skrub.choose_from` over three hyperparameters combined with `.skb.make_grid_search()`. Data column indicates whether the pipeline was run against real competition data or synthetic data matched to the original schema (real Kaggle data was not downloadable in the development environment); this is disclosed per-pipeline rather than uniformly, since it materially affects the strength of individual correctness claims.

5 Evaluation

We report results for two conditions. *Phase 1* evaluates hand-verified reference conversions, written to establish ground truth and to validate the analyzer/validator independently of any LLM. *Phase 2* evaluates the fully automated `converter.py` pipeline, using the `qwen3-coder-next` open-weight model served via the GWDG/AcademicCloud SAIA endpoint.

	Phase 1 (hand-written)	Run 1	Phase 2 Run 2	Final
Static check pass	10/10	10/10	10/10	10/10
Dynamic check pass	9/10	0/10	5/10	6/10

Table 2: Static and dynamic validation pass rates. Run 1 used the original repair loop (static-only); Run 2 followed the repair-loop fix described in Section 5.1; Final additionally includes the target-column prompt fix described in Section 5.2.

5.1 The Repair-Loop Finding

Phase 2 Run 1 produced syntactically valid Python for all ten pipelines on the first LLM attempt (10/10 static pass), yet every single script failed at runtime (0/10 dynamic pass): a full gap between the two checks. Root cause: the repair loop’s `validate_fn` was wired only to `static_check()`; because none of the ten scripts failed that check, the loop never fired, regardless of whether the script would actually run. The most common single failure (4 of 10 pipelines) was a DataOps var declared with a placeholder default value, `skrub.var("df", None)`: `skrub` evaluates a var’s default eagerly for interactive preview purposes, so `None` guarantees an `AttributeError` on the very first operation. We fixed this at two levels: (1) `validator.py` gained a `runtime_check()` that actually executes the candidate script and is now combined with the static check inside the repair loop’s `validate_fn`, and (2) `prompts.py`’s API ref-

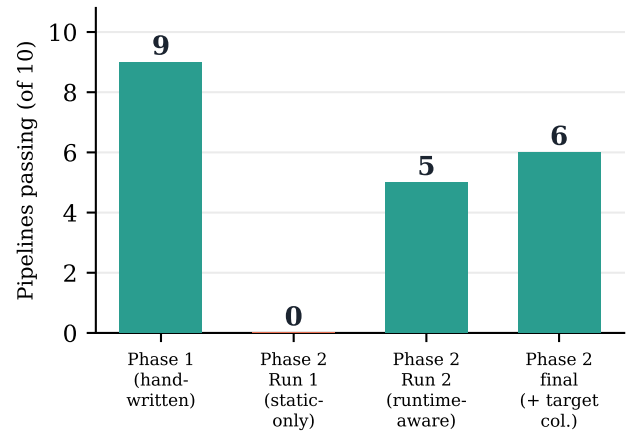


Figure 1: Dynamic-validation pass rates across evaluation stages. The progression from 0/10 to 6/10 reflects two framework fixes, each surfaced by end-to-end evaluation rather than code inspection.

erence was extended with an explicit rule against placeholder default values, among other corrected rules (missing imports, correct `.skb.concat()` usage, correct `AggJoiner` semantics).

Re-running Phase 2 after these fixes raised the dynamic pass rate from 0/10 to 5/10 (Table 2, Figure 1). Disaggregating this improvement is itself an important part of the finding: of the four pipelines broken solely by the placeholder-default bug, three passed cleanly on the first attempt after the prompt fix, with no repair round needed at all, meaning the improvement there is attributable to better *upfront* prompting, not to the repair loop’s self-correction ability. Of six pipelines that did trigger a repair round in Run 2, only one was actually resolved by it (Figure 3). The remaining five repair attempts fed the LLM a correct, specific runtime error message and the LLM’s retry still did not produce working code, typically because the underlying bug was the LLM inventing a DataOps method

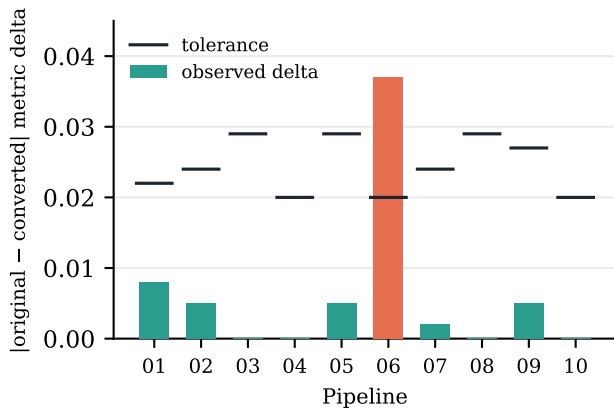


Figure 2: Phase 1 per-pipeline metric deltas between the original and converted pipelines, against each pipeline’s tolerance ($\max(0.02, 0.03 \times |\text{original}|)$, black markers). Only example 06 (stacking) exceeds tolerance, for the structural reason discussed in Section 5.6.

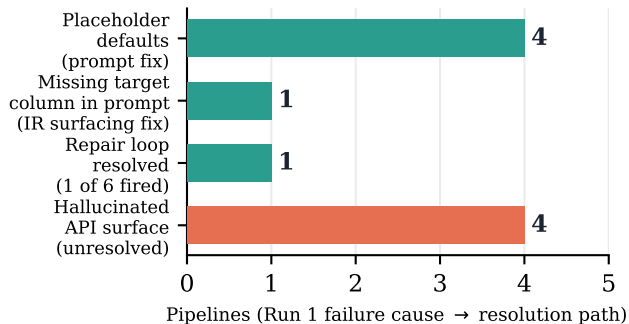


Figure 3: Resolution paths for the ten Run 1 failures. Most recovered pipelines were fixed by improving what the LLM is told upfront (prompt rules; explicit target-column surfacing), not by the LLM correcting itself from error feedback; hallucinated-API failures remain unresolved.

or class that does not exist (`skrub.FrequencyEncoder`, a module-level `skrub.concat`, or non-existent prediction-extraction methods on an estimator DataOp node), a class of error that a single corrective message does not reliably resolve within three rounds. We therefore report the repair loop, once correctly wired to fire, as *necessary but not currently sufficient*: it recovers gracefully from simple errors but has a low observed success rate against hallucinated-API errors specifically.

5.2 The Target-Column Finding

One further failure had a qualitatively different root cause worth separating from the hallucination class. On the Credit Fraud multi-table pipeline, the LLM guessed the target column name from a hardcoded candidate list (`["target", "label", "is_fraud", "fraud"]`) rather than using the exact name (`fraud_flag`) already present in the inter-

mediate representation it was given: the information existed but was buried in a JSON blob rather than stated as an explicit instruction. We fixed this by surfacing `target_column` as an unambiguous directive in the prompt text, re-ran only that pipeline, and it passed, raising the final automated pass rate to 6/10. The general lesson: for information the converter must get exactly right, presence in the IR is not enough; the prompt must state it explicitly.

5.3 Remaining Phase 2 Failures

Four pipelines remain unresolved after all fixes. All four share the same failure class: the LLM invents DataOps API surface that does not exist (a `FrequencyEncoder` class, a module-level `concat`, function-vs-DataOp confusion in a nested helper, and prediction-extraction methods on an estimator node). These reflect genuine gaps in the model’s knowledge of `skrub`’s comparatively young and sparsely documented DataOps API, which stronger few-shot examples, retrieval over real `skrub` documentation at generation time, or a larger code-generation model might close; we discuss these directions in Section 8.

5.4 Cross-Model Comparison

To probe whether the hallucination failure mode is specific to our canonical model or general, we ran an indicative comparison across four open-weight models available through the GWDG SAIA endpoint, on three pipelines chosen to span difficulty: 01 (single-table, easy), 03 (multi-table join, medium), and 06 (stacking ensemble, hard). Two models are code-specialised (`qwen3-coder-next`, `devstral-2-123b`) and two are general-purpose (`gpt-oss-120b`, `glm-4.7`). All models used identical tolerances and repair settings. Results are shown in Table 3 and Figure 4.

Model	Type	Pass/3	Repairs
qwen3-coder-next	code-spec.	2/3	1.0
devstral-2-123b	code-spec.	2/3	1.0
gpt-oss-120b	general	3/3	0.0
glm-4.7	general	3/3	0.0

Table 3: Cross-model comparison on three pipelines (easy/medium/hard). Mean repair rounds shown. All four solve the easy and medium pipelines; they diverge only on the hard stacking pipeline.

The result is counter-intuitive: the two general-purpose models outperformed the two code-specialised ones, and the divergence is entirely on the hard stacking pipeline. The two code-specialised models attempted to implement stacking *natively* in DataOps, hallucinating non-existent primitives (for example `.skb.fit(y=y)` and estimator-internal accessors) and exhausting all three repair rounds. The two general models instead took the pragmatic route of wrapping `scikit-learn`’s intact `StackingRegressor` in a single `.skb.apply()` call, which we verified is bit-identical to the original pipeline (see

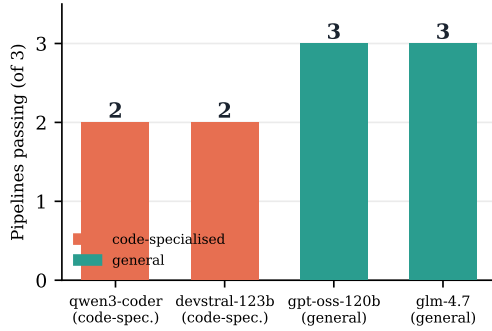


Figure 4: Pipelines passing (of three) per model. Counter-intuitively, the two general-purpose models outperform the two code-specialised ones.

below). The hallucination failure mode is therefore not universal but concentrated at the intersection of code-specialised models and unfamiliar API surface; for this task, instruction-following conservatism about an unfamiliar API appears more valuable than code fine-tuning. This is an indicative three-pipeline, single-run probe rather than a full benchmark, and fresh generation is non-deterministic.

5.5 A Better Conversion for Stacking

The cross-model probe surfaced a finding about our own reference conversions. For pipeline 06, our hand-verified Phase 1 reference decomposes the stacking ensemble into separate DataOps branches with a meta-learner (the idiomatic “native DataOps” approach), and this is the source of the $\Delta = 0.037$ gap in L1 above: those branches are each fit on the full training set, giving the meta-learner in-fold rather than out-of-fold predictions. The pragmatic wrap-in-apply approach the general models found instead keeps `StackingRegressor` intact and delegates its internal out-of-fold cross-validation to scikit-learn; we verified directly (same seeds and data) that this yields predictions bit-identical to the original ($\Delta = 0$). In other words, the general models produced a *better* conversion of this pipeline than our own hand-written reference did. This refines rather than contradicts L1: skrub genuinely has no primitive for expressing out-of-fold stacking as native DataOps graph nodes, but the correct conversion is not to decompose the ensemble at all: it is to apply the intact composite estimator, letting scikit-learn own the cross-validation. We retain the original reference and disclose this rather than silently revising it, since the discovery itself is part of the result.

5.6 Known Framework Limitations

Beyond the repair-loop finding, four structural limitations are disclosed: (L1) skrub’s current DataOps API has no primitive for expressing out-of-fold stacking as native DataOps graph nodes, so a *decomposed* conversion of `StackingRegressor` (separate branches plus a meta-learner) is not numerically equivalent to the original (ob-

served on example 06, $\Delta = 0.037$ against a tolerance of 0.020). As discussed in Section 5.5, the correct conversion is to apply the intact composite estimator rather than decompose it, which is bit-identical to the original; this refines the limitation to native decomposition specifically rather than stacking in general; (L2) the AST-based fallback analyzer does not currently visit inline `lambda` expressions passed to `.apply()`, only named function definitions; (L3) the intermediate representation models only linear join chains between tables, not a general join graph; (L4) skrub’s default automatic text-encoding path (`TableVectorizer` routing through `MinHashEncoder`) was found to underperform a hand-tuned `TfidfVectorizer` by roughly 0.1 AUC on word-vocabulary text classification (example 08), a structural limitation of character-n-gram hashing rather than a tuning issue, with the semantically closer alternative (`skrub.TextEncoder`, sentence-transformer embeddings) unavailable in this environment due to a missing optional dependency.

6 Generalizability

The framework is general by construction, not specialized to the ten evaluated pipelines: nothing in `analyzer.py`, `converter.py`, or `validator.py` references any specific dataset, column name, or estimator, and the CLI accepts an arbitrary source script. Any pipeline exposing a scikit-learn `Pipeline/ColumnTransformer` object is handled by direct object introspection; imperative scripts without one fall back to the AST-based analysis path. The evaluated corpus was chosen to probe this generality along several axes at once: task type (binary, multi-class, regression), table structure (single-table, two-table joins, wide 130–200-column frames), estimator family (linear models, random forests, gradient boosting [7], calibrated classifiers, stacking ensembles), and feature types (numeric, categorical, datetime, free text). Within that span, conversion fidelity was high wherever a faithful DataOps counterpart exists (Figure 2).

At the same time, we distinguish being *general* from being *validated*: empirical evidence covers exactly these ten pipelines, and we can name the structural patterns beyond them where the framework would currently degrade. Pipelines whose tables form a genuine join graph (three or more tables joined non-linearly) exceed what the IR’s linear-chain table model can represent; imperative feature engineering expressed through inline `lambdas` is invisible to the AST fallback; and any pipeline built on out-of-fold ensembling inherits the stacking gap of Section 5.6 regardless of its domain. For the automated path specifically, generalization is bounded less by the framework than by the LLM’s knowledge of the DataOps API surface: the four unresolved failures all stem from hallucinated API calls, a failure mode that any structurally novel pipeline could trigger. We therefore characterize the framework as broadly applicable across common tabular pipeline shapes, with precisely enumerated (rather than open-ended) boundaries.

Stratum (deem-data/stratum) is a drop-in, Rust-accelerated execution backend for the identical DataOps operator API (import stratum as skrub), not a separate target syntax. Every generated script in this project uses a try: import stratum as skrub / except ImportError: import skrub guard, and stratum’s public API surface was confirmed to re-export the exact operators used across all ten examples. Execution-level verification (actually building and running against stratum’s Rust backend) was not attempted in this project’s development environment, which lacked a Rust toolchain and Python 3.12; the validator’s dynamic check already supports this via a use_stratum flag that sets STRATUM_RUST_BACKEND=1 for the harness subprocess, so no further code changes are required to complete this verification once stratum is built.

7 Installation and Reproduction

The project has no unusual system dependencies beyond a Rust toolchain for the optional stratum backend. To reproduce this work on a new system:

1. **Clone or unpack** the project directory and cd into it.
2. **Create an isolated Python environment** (Python 3.10+):

```
python3 -m venv .venv
source .venv/bin/activate
```

3. **Install dependencies:**

```
pip install skrub scikit-learn xgboost \
pandas numpy pytest category_encoders \
openai
```

On macOS, XGBoost additionally requires the OpenMP runtime: brew install libomp.

4. **Reproduce all reported experiments with one command** (no API key or network required):

```
python run_experiments.py
```

This regenerates every synthetic dataset from its seeded make_data.py script, runs the offline test suite, executes all ten Phase 1 harnesses, re-validates the saved Phase 2 LLM outputs deterministically, and writes per-step logs plus a summary comparison against the reported tables to logs/. EXPERIMENTS.md in the repository maps each experiment to its script, log file, and reported numbers, and states explicitly which single step (fresh LLM generation) is non-deterministic and how its saved artifacts are validated instead.

5. **Run dynamic validation** for any single pipeline individually if desired:

```
cd examples/01_titanic
python harness.py converted_dataops.py
```

6. **Run the automated converter** (requires an API key for either backend):

```
export GWDG_API_KEY=... # free, open-
weight
# or: export ANTHROPIC_API_KEY=...
python -m skrubifier convert \
examples/01_titanic/source_pipeline.py \
--out out.py --backend gwdg
```

A GWDG/AcademicCloud API key is obtained free of charge for academic users via AcademicCloud single sign-on and the KISSKI LLM Service booking form (<https://kisski.gwdg.de>); an Anthropic API key is obtained via <https://console.anthropic.com> and is billed per use.

7. **(Optional) Build stratum** for the accelerated Rust backend:

```
# requires a Rust toolchain and Python
3.12+
curl https://sh.rustup.rs -sSf | sh
pip install maturin
git clone https://github.com/deem-data/
stratum
cd stratum && maturin develop --release
```

No code changes are required afterward: every converted script already attempts import stratum as skrub first.

All large or real competition data files are excluded from version control via .gitignore and are either regenerated synthetically by the scripts in each examples/0N_*/ directory or, for the one real-data pipeline (example 04), included directly since the file is small enough to version.

8 Future Work

Four directions follow directly from the findings. **(1) Grounding the converter against hallucination.** The dominant unresolved failure class is invented DataOps API surface; the most promising mitigations are retrieval-augmented prompting over the actual installed skrub documentation/stubs at generation time, and an API-surface linter in the static check that rejects any skrub.* or .skb.* name not present in the installed package before the script is ever executed, thereby converting hallucinations from runtime failures into precise, repair-friendly messages. **(2) A smarter repair loop.** With a 1-of-6 observed repair success rate, single-shot error feedback is insufficient; multi-turn repair dialogue, or error-class-specific repair templates (e.g., mapping AttributeError on a skrub name directly to the linter’s nearest-valid-API suggestion), are natural next steps. **(3) Closing the structural gaps.** The IR extension from linear join chains to a general join graph, lambda capture in the AST fallback, and an out-of-fold stacking recipe (manual cross_val_predict-style construction inside the DAG) are each well-scoped en-

gineering items already specified in the repository’s limitation notes. **(4) Stratum execution verification.** The remaining step is purely environmental (a Rust toolchain and Python 3.12); the validator already threads the backend switch through. **(5) Broader cross-model evaluation.** Our indicative three-pipeline probe (Section 5.4) compared four open-weight models and found that general-purpose models outperformed code-specialised ones on the hardest pipeline; extending this to all ten pipelines and more model families would establish whether that pattern holds in general, and would quantify how much of the automated pass rate is model-choice-dependent. Because `converter.py` is model-agnostic and the GWDG SAIA endpoint exposes many families, this is a direct extension of the existing infrastructure.

9 Conclusion

Skrubifier demonstrates that a three-stage architecture (deterministic structural analysis, LLM-driven syntax translation, and numerical validation with an automatic repair loop) can convert a meaningful fraction of real scikit-learn tabular pipelines into skrub’s DataOps API without human intervention, while remaining honest about where it currently falls short. The project’s most valuable finding may not be the final 6/10 automated pass rate itself, but the process that produced it: two silent gaps between the framework’s design and its behavior: a repair loop that never fired on runtime errors, and critical IR information never surfaced explicitly to the LLM, were each discovered only through end-to-end dynamic testing, root-caused precisely, fixed, and re-measured (0/10 → 5/10 → 6/10) rather than papered over. The remaining failures are attributable overwhelmingly to LLM hallucination of non-existent DataOps API surface rather than to weaknesses in the analyzer or validator, suggesting that further gains are most likely to come from grounding the converter in the installed API surface and from richer repair strategies (Section 8), rather than from further framework restructuring.

References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] The skrub developers, “skrub: Machine learning with dataframes,” 2024. [Online]. Available: <https://skrub-data.org>
- [3] deem-data, “stratum: A system infrastructure for massive agent-centric ML workloads,” 2026. [Online]. Available: <https://github.com/deem-data/stratum>
- [4] J. S. Chan et al., “MLE-Bench: Evaluating machine learning agents on machine learning engineering,” OpenAI, 2024. [Online]. Available: <https://github.com/openai/mle-bench>
- [5] Kaggle, “Kaggle: Your machine learning and data science community,” [Online]. Available: <https://www.kaggle.com>

- [6] Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen (GWDG), “SAIA / Chat AI: Scalable AI accelerator service,” [Online]. Available: <https://docs.hpc.gwdg.de/services/saia>
- [7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.